

DOCUMENTAÇÃO TÉCNICA

Banco de Dados

RPG de Mesa — Caminho Sem Volta

PostgreSQL 18 · 42 tabelas · 70 migrations
Gerado em 7 de setembro de 2026

Sumário

1. Como ler este documento

Onde o banco roda · convenções que valem para tudo

2. Mapa de ligações

O núcleo em volta de `characters` · ligações por nome

3. Contas e acesso

usuarios · `character_creation_requests` · `character_creation_whitelist`

4. O personagem

`characters` · o JSONB `data` · `player_telas` · `skill_character_override`

5. Mundo e catálogo

`campaigns` · `campaign_gms` · `gods` · `racas` · `passados` · `npcs` · `city_maps` · `lore_notes`

6. Regras de jogo

`classes` · `titles` · `skills` · `skill_niveis` · `progressão` · `classes secretas`

7. Equipamento

`equipamentos` · a hierarquia usada · a hierarquia paralela vazia

8. Lookups fixos

`indole` · `genero` · lookups de skill

9. Fluxos do sistema

`login` · `criação` · `onboarding` · `alteração` · `XP` · `classe secreta` · `upload`

10. Armadilhas conhecidas

o que já quebrou e por quê

1. Como ler este documento

Onde o banco roda

PostgreSQL 18 num container Docker (`docker-compose.yml` na raiz do repositório), publicado na porta **5433**. Há também uma instalação nativa do Postgres no Windows na 5432, mantida como referência e backup — as duas guardam os mesmos dados. A aplicação conecta com o papel `rpg_app_user`; as credenciais ficam fora do repositório.

O esquema completo, em SQL, está em `docs/SCHEMA_CURRENT.sql` — arquivo gerado por `pg_dump`, não editado à mão. As alterações incrementais estão em `database/migrations/`, numeradas de 001 a 070.

Convenções que valem para o banco inteiro

Não existe FOREIGN KEY

Nenhuma. As referências entre tabelas são **por convenção de inteiro**: uma coluna `raca_id` guarda um `racas.id`, e é só isso — o banco não verifica, não impede apagar o alvo, não faz cascata. Toda a integridade fica no código da aplicação.

Consequência prática: apagar um registro referenciado não dá erro nenhum, deixa a referência apontando para o vazio. Quem apaga é responsável por limpar — é por isso que a deleção de skill do catálogo varre `passados`, `titles` e `classes` à mão.

Chave primária: id ou item

Tabelas de entidade usam `id` `INTEGER GENERATED ALWAYS AS IDENTITY`. Tabelas de *lookup* (listas curtas que o mestre edita: tipos, categorias, propriedades) usam `item` `INTEGER` com sequence própria. A diferença é histórica, mas é confiável: se a coluna se chama `item`, é lookup.

Soft delete

Quase toda tabela tem `deleted_at` e `deleted_by`. Registro ativo é `deleted_at IS NULL` — e toda consulta precisa filtrar por isso explicitamente.

Três tabelas são exceção e apagam de verdade: `classe_secreta_revelada`, `class_level_progression`, `skill_niveis` e `npc_acesso_player`. Todas têm índice único *total* (sem `WHERE deleted_at IS NULL`), então uma linha soft-deletada continuaria ocupando a chave e impediria recriar o registro — com o agravante de o culpado estar invisível na listagem.

Auditoria

`created_by`, `updated_by` e `deleted_by` guardam o **e-mail** de quem fez a ação, em TEXT. São preenchidos automaticamente por hooks globais do Sequelize, que leem o usuário autenticado do contexto da requisição.

O preenchimento automático só acontece na escrita **pelo ORM**. `sequelize.query` cru não dispara hook nenhum — quem escreve em SQL passa o autor na mão.

Caminhos de arquivo

Colunas de imagem e documento (`avatar_url`, `foto_url`, `image_url`, `cover_image_url`, `historia_doc_url`, `pdf_url`) guardam o **caminho relativo** dentro de `uploads/`, como `personagens/inari.png`. A URL completa é montada na resposta da API, a partir de `PUBLIC_BASE_URL`. Trocar de host é trocar uma variável de ambiente, não reescrever linhas.

Row Level Security

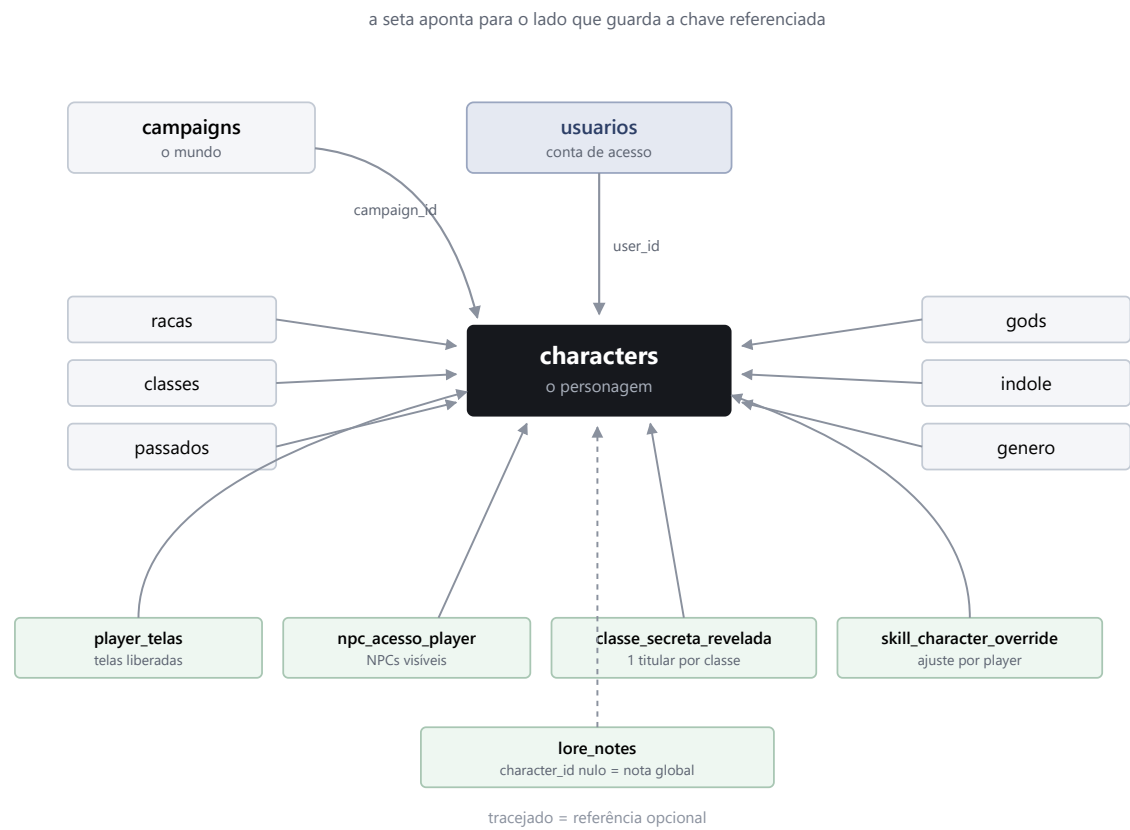
42 tabelas estão com **RLS ligado e quase nenhuma policy** — herança do Supabase. O sistema só funciona porque `rpg_app_user` tem `BYPASSRLS`. A autorização real vive nos *guards* do NestJS. Qualquer conexão com um papel sem `BYPASSRLS` veria a maioria das tabelas vazia e não conseguiria escrever.

Legenda dos rótulos

PK chave primária **REF** referência por convenção **UNIQUE** índice único **NOT NULL** obrigatória **MORTA** sem uso no código

2. Mapa de ligações

`characters` é o centro de gravidade: guarda seis referências para catálogos e é apontada por cinco tabelas satélite. Tudo o mais orbita isso.



O núcleo do esquema. Equipamento, skills e lookups ficam de fora — não se ligam a `characters` por coluna, e sim por cópia dentro do JSONB `data`.

Ligações que não são por id

Uma parte das referências aponta por **nome, em texto**, não por chave. Isso é frágil de propósito ou por herança, mas precisa ser conhecido: renomear o alvo quebra a ligação em silêncio.

ORIGEM	ALVO	OBSERVAÇÃO
<code>classes.starting_skills[]</code>	<code>skills.name</code>	array de texto; a deleção de skill limpa este array
<code>classes.passive_skills[]</code>	<code>skills.name</code>	idem, mas <i>não</i> é limpo na deleção
<code>classes.signature_skill</code>	<code>skills.name</code>	texto simples
<code>skills.raca_vinculada[]</code>	<code>racas.nome</code>	array de texto
<code>skills.required_class</code>	<code>classes</code>	VARCHAR — guarda ora nome, ora id
<code>skill_character_override.skill_name</code>	<code>skills.name</code>	compõe o UNIQUE com <code>character_id</code>
<code>campaign_gms.email</code>	<code>usuarios.real_email</code>	liga mestre a campanha por e-mail
<code>characters.data.skills[]</code>	<code>skills.name</code>	cópia dentro do JSONB, tratada como histórico
<code>*_by, revisado_por, revealed_by</code>	<code>usuarios.real_email</code>	auditoria: e-mail, nunca id

Por que a auditoria guarda e-mail e não id: o registro precisa continuar legível depois que a conta some. Um id apontando para uma linha apagada não diz nada; um e-mail sim.

3. Contas e acesso

usuários

A conta de login. Existe para mestres e jogadores. 7 linhas.

COLUNA	TIPO	NOTAS
id	INTEGER	PK identity
real_email	TEXT	NOT NULL e-mail real; é o login do mestre
username	TEXT	login do jogador. Sem UNIQUE no banco — só o código garante
tipo	TEXT	NOT NULL CHECK 'gm' 'player', default player
ativo	BOOLEAN	NOT NULL default true; o login recusa inativo
password_hash	TEXT	bcrypt. Nulo = pré-registro : o mestre liberou o e-mail, a conta ainda não existe
requires_password_change	BOOLEAN	NOT NULL força o modal de troca no próximo login

← é apontada por characters.user_id ; e por e-mail em campaign_gms.email e em toda coluna de auditoria

Não existe mais auth_user_id : a coluna que ligava ao Supabase Auth saiu na migration 061. Também não existe e-mail sintético {username}@rpg.internal — o jogador entra pelo próprio username.

character_creation_requests

Solicitação de criação submetida por um jogador, aguardando aprovação do mestre.

COLUNA	TIPO	NOTAS
id	INTEGER	PK
email	TEXT	NOT NULL precisa estar pré-registrado em usuarios
username	TEXT	UNIQUE parcial: só entre pendente e aprovado
password_hash	TEXT	bcrypt, gerado na submissão. Transferido para usuarios na aprovação
nome	TEXT	nome do personagem
avatar_url	TEXT	caminho relativo em uploads/pendentes/
indole_id	INTEGER	REF indole.id
genero_id	INTEGER	REF genero.id
aparencia_fisica	TEXT	NOT NULL mín. 100 letras sem espaços
historia_texto	TEXT	mín. 1000 letras, ou o documento abaixo
historia_doc_url	TEXT	Word/PDF em uploads/pendentes/
status	VARCHAR	pendente aprovado rejeitado
rejeitado_motivo	TEXT	preenchido na rejeição
revisado_em / revisado_por	timestampz / TEXT	e-mail do mestre que revisou
campaign_id	INTEGER	REF campaigns.id

O UNIQUE parcial resolve um bug real. Quando o índice era total, um jogador rejeitado não conseguia reenviar: o username continuava ocupado pela própria solicitação recusada e a nova submissão estourava com chave duplicada. Hoje rejeitar devolve o nome.

character_creation_whitelist MORTA

Era a lista de e-mails autorizados a submeter solicitação. Colunas: `id`, `email` (UNIQUE) + soft delete e auditoria. **5 linhas, nenhum código lê ou escreve.**

A autorização virou o pré-registro em `usuarios` — uma linha com `password_hash` nulo. A tabela ainda aparece nos tipos do frontend e a mensagem de erro do login ainda fala em "whitelist". Candidata a remoção.

4. O personagem

characters

A ficha. 3 linhas (uma delas apagada).

COLUMNA	TIPO	NOTAS
id	INTEGER	PK identity
user_id	INTEGER	NOT NULL REF usuarios.id
campaign_id	INTEGER	REF campaigns.id
name	TEXT	NOT NULL
username	TEXT	UNIQUE parcial (WHERE deleted_at IS NULL). Cópia de usuarios.username
level	INTEGER	NOT NULL default 1
data	JSONB	NOT NULL default {} — detalhado a seguir
avatar_url	TEXT	caminho relativo, ex. personagens/inari.png
raca_id	INTEGER	REF racas.id — nulo até o onboarding
classe_id	INTEGER	REF classes.id — a classe inicial
passado_id	INTEGER	REF passados.id
deus_id	INTEGER	REF gods.id — nulo se pulou a etapa
indole_id	INTEGER	REF indole.id
genero_id	INTEGER	REF genero.id
onboarding_completo	BOOLEAN	NOT NULL falso redireciona para /onboarding
status	TEXT	NOT NULL CHECK 'vivo' 'morto'
aparencia_fisica	TEXT	
historia_texto	TEXT	
historia_doc_url	TEXT	caminho relativo

→ aponta para usuarios, campaigns, racas, classes, passados, gods, indole, genero

← é apontada por classe_secreta_revelada, skill_character_override, npc_acesso_player, player_telas, lore_notes

O JSONB data

Metade da ficha mora aqui, sem esquema declarado em lugar nenhum. Vale conhecer o conteúdo:

CHAVE	O QUE GUARDA
atributos	a soma usada em jogo: base + bônus do passado
atributos_base	os 10 pontos que o jogador distribuiu no onboarding
atributos_bonus_passado	a parcela vinda do passado escolhido
classes	[{name, nivel, xp, skillPoints}] — progressão por classe
classPoints	pontos de classe ainda não gastos
skills / titles	skills e títulos concedidos, copiados por nome
equipamentos_iniciais	escolha da etapa 6, com peso por item
inventario	itens livres, sem peso

CHAVE	O QUE GUARDA
adventureNotes	notas de aventura escritas pelo mestre
pendingChangeRequest	pedido de alteração aguardando revisão (há índice parcial em cima disso)
avatarFocalPoint / modalHeroPosition	enquadramento da imagem, ajustado pelo mestre
xp	XP do personagem — distinto do XP por classe
deusEtapaConcluida	marca a etapa 5 como vista, mesmo se pulada

PATCH /api/personagens/:id substitui o data inteiro. Mandar um objeto parcial apaga todo o resto — atributos, classes, skills, inventário — sem erro nenhum. Ler, alterar a chave, devolver o objeto completo.

player_telas

Quais telas do menu cada personagem pode abrir. **UNIQUE** (character_id, tela) . **9 linhas**.

A lista de telas liberáveis é **fixa no código**, não no banco: cidade , classes , deuses , equipamentos , notas , npcs , racas , skills , titulos . Valor desconhecido é descartado em silêncio na gravação. O mestre sempre recebe todas, sem consultar a tabela.

PUT /api/player-telas/admin/:characterId substitui o conjunto inteiro — apaga tudo e reinsere.

skill_character_override

Permite ao mestre dar a um jogador específico um dano base ou multiplicador diferente do catálogo. **UNIQUE** (skill_name, character_id) .

COLUNA	TIPO	NOTAS
skill_name	TEXT	referência a skills.name — por nome
character_id	INTEGER	REF characters.id
damage_base_override	TEXT	notação de dado, ex. 2d8
multiplicador_override	TEXT[]	mesmo formato de skills.multiplicador_atributo

5. Mundo e catálogo

campaigns · campaign_gms

As campanhas são os "mundos" da tela inicial. **1 campanha** cadastrada.

COLUNA	TIPO	NOTAS
id	INTEGER	PK
slug	VARCHAR	UNIQUE é o que aparece em <code>/mundo/:slug</code>
name / description	VARCHAR / TEXT	
cover_image_url	TEXT	caminho relativo
is_active	BOOLEAN	NOT NULL só as ativas aparecem na listagem pública

`campaign_gms` liga um `campaign_id` a um `email` de mestre. **Sem UNIQUE** — nada impede duplicar o mesmo mestre na mesma campanha.

← é apontada por `characters.campaign_id`, `character_creation_requests.campaign_id`, `campaign_gms.campaign_id`

gods

Panteão. **21 linhas**. Colunas de conteúdo: `name` (UNIQUE), `title`, `description`, `short_description`, `dogma`, `anatema`, `weapons`, `image_url` (caminho relativo), `data` JSONB.

→ aponta para `indole.id` via `indole_id`

← é apontada por `characters.deus_id`

Carrega ainda `image_path` e `indole` (texto), anteriores à normalização da migration 063. Sem uso.

racas

Raças jogáveis. **4 linhas**.

COLUNA	TIPO	NOTAS
nome	TEXT	NOT NULL
descricao	TEXT	visível a todos
lore	TEXT	só o mestre vê — a listagem pública devolve <code>null</code>
foto_url	TEXT	caminho relativo
habilidades	JSONB	NOT NULL [{nome, descricao}]
atributos_bonus	JSONB	NOT NULL [{atributo, valor}]

← é apontada por `characters.raca_id`, `npcs.raca_id`, e por nome em `skills.raca_vinculada[]`

passados

Origens do personagem. Cada passado concede skills e títulos. **7 linhas**.

COLUNA	TIPO	NOTAS
nome / descricao	VARCHAR / TEXT	
foto_url	TEXT	caminho relativo
skill_ids	INTEGER[]	NOT NULL REF skills.id
titulo_ids	INTEGER[]	NOT NULL REF titles.id
atributo_bonus	JSONB	bônus somado aos atributos no onboarding

As skills e títulos do passado **não são copiados** para o personagem. O dashboard os lê do catálogo na hora de exibir — então editar o passado muda retroativamente o que os jogadores daquele passado enxergam.

npcs · npc_acesso_player

NPCs e quem pode vê-los. Ambas vazias por enquanto.

npcs: nome, raca_id (REF racas.id), descricao, foto_url.

npc_acesso_player: UNIQUE (npc_id, character_id) total — por isso revogar acesso é **hard delete**. O jogador só vê o que estiver listado; o mestre vê tudo.

city_maps

Mapas da cidade. **1 linha**.

name (UNIQUE), map_reference, description, data
JSONB. Não se liga a nenhuma outra tabela.

lore_notes

Notas de lore publicadas pelo mestre. Vazia.

title, subtitle, content (NOT NULL, default ''),
ordem, pdf_url.

character_id nulo = nota global (todos veem);
preenchido, só aquele personagem.

6. Regras de jogo

classes

Classes jogáveis. 29 linhas.

COLUNA	TIPO	NOTAS
name	TEXT	UNIQUE
tier	TEXT	CHECK: Base, Híbrida ou Hidden
max_level	INTEGER	NOT NULL default 20
requirements	JSONB	NOT NULL default {}
stat_bonuses	JSONB	NOT NULL default {}
starting_skills	TEXT[]	NOT NULL default {} — nomes de skill
passive_skills	TEXT[]	nullable — nomes de skill
signature_skill	TEXT	nome da skill de assinatura
requer_deus	BOOLEAN	NOT NULL
is_secret	BOOLEAN	NOT NULL classes secretas não aparecem no onboarding

← é apontada por characters.classe_id, classe_secreta_revelada.classe_id, class_level_progression.classe_id, titles.classe_secret_id

classe_secreta_revelada

Qual personagem detém cada classe secreta. UNIQUE (classe_id) garante exclusividade — um titular por classe.

Colunas: classe_id, character_id, revealed_at, revealed_by (e-mail do mestre).

Marcar o personagem como **morto** apaga automaticamente os registros dele, liberando as classes. Personagens mortos não aparecem na lista de seleção ao revelar.

O UNIQUE é **total**, não parcial. Uma linha soft-deletada continuaria ocupando a chave e impediria revelar a classe de novo — invisível na listagem. Por isso revogar é **hard delete**.

titles

Títulos. 20 linhas. name UNIQUE, tier, description.

COLUNA	TIPO	NOTAS
bonuses	JSONB	NOT NULL default {}
requirements	JSONB	NOT NULL default {}
skill_ids	INTEGER[]	NOT NULL REF skills.id
is_hidden	BOOLEAN	oculta os requisitos do jogador
classe_secret_id	INTEGER	REF classes.id — preenchido, o título só aparece para quem tem aquela classe revelada

skills

Catálogo de habilidades. 66 linhas. name UNIQUE.

COLUNA	TIPO	NOTAS
description	TEXT	NOT NULL sem descrição grava string vazia, nunca null
damage_base	TEXT	notação de dado, ex. 1d8 , 2d6+3
multiplicador_atributo	TEXT[]	expressões que escalam o dano, ex. {"2d8 + Destreza"}
effect_description	VARCHAR	
custo / cooldown / range	INTEGER / INTEGER / TEXT	
skill_tipo_item	INTEGER	REF skill_tipo.item
skill_categoria_item	INTEGER[]	REF skill_categoria.item
skill_tipo_dano_item	INTEGER[]	REF skill_tipo_dano.item
skill_natureza_item	INTEGER	REF skill_natureza.item
raca_vinculada	TEXT[]	nomes de raça
required_class	VARCHAR	ora nome, ora id — inconsistente
nivel_minimo_classe	INTEGER	inerte: o dashboard lê, mas está nulo em todas as linhas e nenhuma rota escreve

A tabela ainda carrega damage_modifier , damage_type , cost , is_secret e required_class_id , anteriores à migration 047. Nenhum código lê ou escreve, mas as três primeiras têm dados e continuam vindo na resposta da API porque a versão antiga fazia SELECT * .

skill_niveis

Evoluções de nível 2 e 3 de uma skill. UNIQUE (skill_id, nivel) total → o DELETE é hard. Vazia.

Cada campo *_override (nome , damage_base , multiplicador , effect_description) substitui o da skill base quando preenchido. damage_multiplier_pct é um bônus percentual de dano.

As duas tabelas de progressão

É fácil trocar uma pela outra — têm propósitos diferentes.

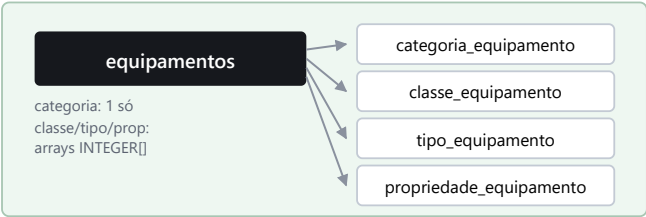
TABELA	ESCOPO	COLUNAS E USO
level_progression	personagem	level (UNIQUE), tier, multiplier, xp_required_next, xp_total_accumulated . 27 níveis. É a tabela percorrida para decidir o nível a partir do XP acumulado
class_level_progression	classe	classe_id, nivel, xp_necessario, com UNIQUE (classe_id, nivel) . Alimenta data.classes[].xp . Vazia

O código antigo consultava colunas que não existem em level_progression (nivel , xp_necessario — que são da outra tabela). O resultado é que conceder XP nunca subia o nível de ninguém. Nas duas o UNIQUE é total, então apagar um nível é hard delete.

7. Equipamento

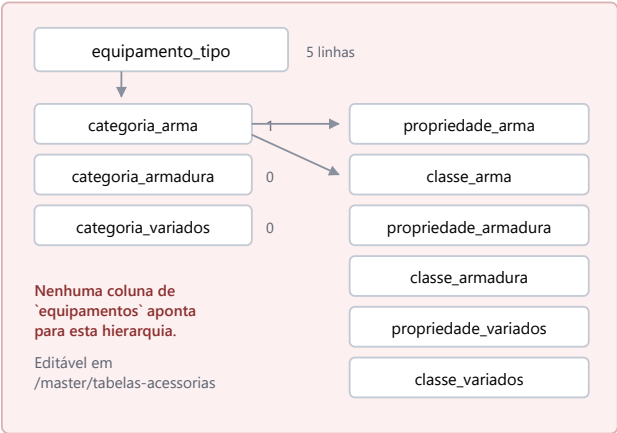
Existem **duas hierarquias paralelas** de classificação de equipamento no banco. Só uma é usada. Saber qual poupa muito tempo.

EM USO — 15 equipamentos cadastrados



A hierarquia da esquerda é a que o cadastro de equipamento realmente usa.
A da direita tem tela própria, backend próprio e 1 linha de dado.

PARALELA — praticamente vazia



As duas hierarquias de classificação. Ao mexer em equipamento, a da esquerda é a que importa.

equipamentos

Armas, armaduras e itens. **15 linhas**. Antes se chamava `armas`.

COLUNA	TIPO	NOTAS
nome	VARCHAR	NOT NULL
dano	VARCHAR	NOT NULL item que não é arma grava string vazia , nunca null
peso / valor	NUMERIC	kg e moedas
descricao_equipamento	VARCHAR	
pre_requisitos	VARCHAR	
categoria_equipamento_item	INTEGER	uma só REF categoria_equipamento.item
classe_equipamento_item	INTEGER[]	NOT NULL lista
tipo_equipamento_item	INTEGER[]	NOT NULL lista
propriedade_equipamento_item	INTEGER[]	NOT NULL lista

Assimetria fácil de inverter: categoria é *uma só* (coluna `integer`), enquanto classe, tipo e propriedade são *listas* (`integer[]`). A documentação do projeto descrevia o contrário até a migração do módulo.

Nenhuma tabela aponta para `equipamentos`. O que o personagem carrega é uma cópia dentro de `characters.data.equipamentos_iniciais` e `data.inventario`.

categoria Equipamento

Topo da hierarquia. **8 linhas.**

`item` PK, `descricao`, `icone`. Seed: Armadura, Exploração, Cura, Cosmético, Utilitário, Armas.

É apontada por `equipamentos`, `tipo_equipamento.categoria_item` e `propriedade_equipamento.categoria_item`.

classe Equipamento

Dimensão independente. **8 linhas.**

Não é pai de ninguém. Ex.: Simples, Marcial, Exótica, Couro.

tipo Equipamento

Filho de categoria. **10 linhas.**

Ex.: Corpo a corpo, Longo alcance. Tem também uma coluna `classe_item` herdada, vazia em todas as linhas e não mapeada no código.

propriedade Equipamento

Filho de categoria. **26 linhas.**

Ex.: Ágil, Área. Mesma estrutura de `tipo_equipamento`, mesma `classe_item` vazia.

O `item` vem da sequence do banco. A versão antiga calculava `MAX(item)+1` numa consulta à parte: duas idas ao banco por inserção, duas criações simultâneas escolhendo o mesmo número, e a sequence nunca avançando. Ver a armadilha nº 2 na última seção.

8. Lookups fixos

indole

Alinhamento. **5 linhas**, seed fixo.

`id` PK, `codigo` (UNIQUE), `descricao`.

`bom`, `neutro-bom`, `neutro`, `neutro-ruim`, `ruim`.

Apontada por `characters`, `gods` e `character_creation_requests`.

genero

Gênero e pronome. **3 linhas**, seed fixo.

`id` PK, `codigo` (UNIQUE), `descricao`, `pronome`.

`ela`, `ele`, `outro`.

Apontada por `characters` e `character_creation_requests`.

Lookups de skill

Todos seguem o padrão `item` INTEGER PK + `descricao` + soft delete + auditoria. Editáveis em `/master/skills`.

TABELA	LINHAS	REFERENCIADA POR
<code>skill_tipo</code>	4	<code>skills.skill_tipo_item</code> — valor único
<code>skill_categoria</code>	6	<code>skills.skill_categoria_item[]</code> — array
<code>skill_tipo_dano</code>	8	<code>skills.skill_tipo_dano_item[]</code> — array
<code>skill_natureza</code>	3	<code>skills.skill_natureza_item</code> — Ativa, Passiva, Assinatura

Tabelas acessórias de equipamento

A hierarquia paralela da seção anterior. Todas com `item` PK + `descricao`, soft delete e auditoria. Editáveis em `/master/tabelas-acessorias`.

TABELA	LINHAS	PAI
<code>equipamento_tipo</code>	5	— (seed: Arma, Armadura, Variados)
<code>categoria_arma</code>	1	<code>equipamento_tipo_item</code>
<code>categoria_armadura</code>	0	<code>equipamento_tipo_item</code>
<code>categoria_variados</code>	0	<code>equipamento_tipo_item</code>
<code>propriedade_arma</code>	0	<code>categoria_arma_item</code>
<code>classe_arma</code>	0	<code>categoria_arma_item</code>
<code>propriedade_armadura</code>	0	<code>categoria_armadura_item</code>
<code>classe_armadura</code>	0	<code>categoria_armadura_item</code>
<code>propriedade_variados</code>	0	<code>categoria_variados_item</code>
<code>classe_variados</code>	0	<code>categoria_variados_item</code>

9. Fluxos do sistema

9.1 Autenticação

JWT próprio, emitido pelo backend. Não há sessão no servidor: sair é apagar o que está guardado no navegador.

- 1 **POST /api/auth/login** com { `identificador`, `senha` }. O `identificador` é o *username* do jogador ou o *e-mail real* do mestre.
lê: `usuarios.username`, `usuarios.real_email`, `usuarios.password_hash` (bcrypt)
- 2 Recusa se `ativo = false`, ou se `password_hash` for **nulo** — nulo significa pré-registro, conta ainda não criada.
- 3 Resposta: { `tokenAcesso`, `tipo`, `precisaTrocarSenha`, `usuario` }. O `tipo` (`gm` / `player`) viaja **dentro do JWT** — é daí que sai a decisão de quem é mestre.
JWT: { `usuariold`, `email`, `tipo` }
- 4 O navegador guarda `rpg-mesa.token` e `rpg-mesa.auth-meta` no `localStorage`. A sessão expira em 24 h, verificado no router guard e no store.
- 5 Cada requisição leva `Authorization: Bearer <token>`. O `JwtAuthGuard` valida e publica o usuário no contexto (AsyncLocalStorage) — de onde os hooks do Sequelize tiram o autor da auditoria.
- 6 Se `requires_password_change` for verdadeiro, aparece o modal obrigatório de troca: no `DashboardView` para jogador, no `MasterPanelView` para mestre.
grava: `usuarios.password_hash`, `usuarios.requires_password_change = false`

9.2 Criação de personagem

- 1 **Mestre pré-registra o e-mail** em `/master/usuarios`. Sem isso a submissão é recusada.
grava: `usuarios` (`password_hash` nulo)
- 2 Jogador abre `/`, escolhe o mundo, cai em `/mundo/:slug` e abre "Criar Novo Personagem".
- 3 Sobe o avatar por **POST /api/character-creation-requests/upload-avatar** — rota **pública**, porque quem submete ainda não tem conta.
grava: `uploads/pendentes/<arquivo>.png`
- 4 Submete **POST /api/character-creation-requests**. O backend confere pré-registro, formato e unicidade do username, e as regras de tamanho: aparência ≥ 100 letras, história ≥ 1000 letras ou documento.
grava: `character_creation_requests` (`password_hash` em bcrypt)
- 5 Jogador vê "Aguardando aprovação" — **sem login automático**.
- 6 Mestre vê o sino em `/master` e abre `/master/logins`.
lê: `character_creation_requests` WHERE status = 'pendente'
- 7 **Aprovando:** o pré-registro em `usuarios` é preenchido com o hash já pronto, e o personagem é criado. Se a criação do personagem falhar, a conta é desfeita para não ficar órfã.
grava: `usuarios.password_hash` · `characters`
- 8 **Rejeitando:** grava o motivo. O username volta a ficar livre — o índice único cobre só `pendente` e `aprovado`.

A senha nunca é recuperável a partir do banco. Antes era AES-256-CBC reversível, porque o texto puro era necessário para criar a conta no Supabase Auth — e a chave de criptografia estava no valor padrão do código. Como a conta agora nasce no próprio backend, basta transferir o hash.

9.3 Onboarding — as 6 etapas

Depois de aprovado, o jogador entra e passa pelo onboarding antes do dashboard. `DashboardView` detecta `onboarding_completo = false` e redireciona. Mestres nunca são redirecionados.

#	ETAPA	ENDPOINT	ONDE GRAVA
1	Raça	PATCH <code>/personagens/:id/escolher-raça</code>	<code>characters.raca_id</code>
2	Classe	PATCH <code>/personagens/:id/escolher-classe</code>	<code>characters.classe_id</code> + <code>data.classes</code> (2 pontos de skill)
2b	Skill inicial	POST <code>/personagens/:id/escolher-skill-inicial</code>	<code>data.skills</code> – gasta 1 ponto, sobe o nível da classe
3	Passado	PATCH <code>/personagens/:id/escolher-passado</code>	<code>characters.passado_id</code>
4	Atributos	PATCH <code>/personagens/:id/definir-atributos</code>	<code>data.atributos_base</code> , <code>_bonus_passado</code> , <code>atributos</code>
5	Deus	PATCH <code>/personagens/:id/escolher-deus</code>	<code>characters.deus_id</code> – pode ser pulada
6	Equipamentos	PATCH <code>/personagens/:id/concluir-onboarding</code>	<code>data.equipamentos_iniciais</code> + <code>onboarding_completo = true</code>

A etapa 2b só aparece se a classe tiver `starting_skills`. O jogador pode voltar livremente às etapas já concluídas pelo stepper do topo.

Capacidade de carga (etapa 6): peso máximo = $2 + \text{força} \times 2$, onde a força já inclui o bônus do passado. O backend valida na conclusão — não dá para passar do limite pelo cliente.

Atributos: 10 pontos distribuíveis. O `data` guarda as três parcelas separadas de propósito, para o bônus do passado poder ser recalculado sem perder o que o jogador escolheu.

9.4 Pedido de alteração da ficha

- 1 Jogador abre "Gerenciar Personagem" no dashboard e pede a mudança (nome, avatar, aparência, história).
grava: `characters.data.pendingChangeRequest` — e sobe arquivo por `/personagens/:id/upload-avatar`
- 2 O pedido aparece para o mestre. Há índice parcial em `data ? 'pendingChangeRequest'` justamente para essa listagem ser barata.
lê: `GET /personagens/admin/solicitacoes`
- 3 Mestre revisa. Aprovando, os campos pedidos são aplicados ao personagem e o `pendingChangeRequest` é removido.
grava: `characters` (campos aprovados)

9.5 Progressão e XP

- 1 Mestre concede XP **ao personagem** (PATCH `/personagens/admin/:id/atribuir-xp-personagem`) ou **a uma classe** (`.../atribuir-xp`). São coisas distintas.
- 2 Para o personagem: o XP é somado a `data.xp` e o nível é recalculado percorrendo `level_progression` por `xp_total_accumulated`.
lê: `level_progression` · grava: `characters.level`, `data.xp`
- 3 Mestre também concede pontos de classe e de skill, que o jogador gasta depois.
grava: `data.classPoints`, `data.classes[].skillPoints`
- 4 Jogador gasta: adquire classe nova (POST `/personagens/:id/escolher-classe`), converte ponto de classe em ponto de skill (`.../levar-classe`) ou distribui atributos.

9.6 Classes secretas

- 1 A classe é marcada com `is_secret = true`. Some do onboarding e da listagem dos outros jogadores.
- 2 Mestre revela em `/master/classes-secretas`: escolhe a classe e o personagem (só **vivos** aparecem).
grava: `classe_secreta_revelada` (UNIQUE `classe_id` — um titular por classe)
- 3 Títulos com `classe_secreta_id` preenchido passam a aparecer para aquele jogador, e só para ele.
- 4 Revogar é **hard delete** — o UNIQUE total não admite soft delete.

- 5 Marcar o personagem como **morto** (`PATCH /personagens/admin/:id/status`) apaga todos os registros dele, liberando as classes automaticamente.

grava: `characters.status` · apaga: `classe_secreta_revelada`

9.7 Liberação de telas e NPCs

- 1 Mestre define em `/master/telas` quais telas o personagem enxerga. O `PUT` substitui o conjunto inteiro.
grava: `player_telas` (apaga tudo e reinsere)
- 2 Em `/master/npcs`, aba de acessos, marca quais personagens veem cada NPC.
grava/apaga: `npc_acesso_player` (hard delete)
- 3 O jogador consulta `GET /player-telas/me?characterId=` e `GET /npcs/player?characterId=`. As duas rotas checam que o personagem é dele.

9.8 Upload de arquivos

- 1 O arquivo sobe pelo backend — **nunca direto do navegador para o storage**, como era no Supabase.
- 2 Imagem passa por `sharp`: rotaciona pelo EXIF, cabe em 800×800 e vira PNG com compressão máxima. Limite de 5 MB na entrada.
- 3 O nome é higienizado (sem acento, minúsculo) e recebe sufixo numérico **só em colisão real com conteúdo diferente**. Reenviar o mesmo arquivo reaproveita o que já está lá, em vez de duplicar.
grava: `uploads/<subpasta>/<nome>.png`
- 4 A rota devolve `{ path, publicUrl }`. O banco guarda o **path relativo**; a URL completa é remontada em toda leitura.

Subpastas: `gods`, `maps`, `racas`, `passados`, `npcs`, `campanhas`, `lore`, `personagens` (avatar e história de personagem existente) e `pendentes` (solicitação de criação).

Todas as rotas de upload exigem mestre, **exceto** as de `character-creation-requests` (públicas por necessidade) e as de `personagens/:id/*`, que exigem ser dono do personagem.

10. Armadilhas conhecidas

Cada uma destas já causou um bug real neste projeto. Todas falham em silêncio — nenhuma dá erro na hora.

1. Coluna NOT NULL recebendo null explícito

O default do Postgres **só se aplica quando a coluna é omitida do INSERT**. Passar `null` explicitamente viola a constraint. Já aconteceu em `classes.starting_skills`, `equipamentos.dano`, `skills.description`, `titles.bonuses` e `lore_notes.content`. Ao mexer num módulo, confira toda coluna NOT NULL com default.

2. Sequences dessincronizadas

O código antigo escolhia a chave com `MAX(item)+1` e mandava o valor explícito. A sequence nunca era chamada e ficou parada no início enquanto os dados avançavam. Ao passar a deixar o banco gerar a chave, a primeira inserção estourou com *duplicate key*. Aconteceu em `skill_tipo_dano`, parada em 1 com registros até o item 8.

A migration **067** percorre toda sequence ligada a uma coluna e a adianta para o maior valor gravado. É idempotente — rode de novo antes de mexer em qualquer módulo que ainda escolha chave na mão.

Ao escrever essa verificação: uma sequence **nunca usada** (`is_called = false`) devolve o próprio `last_value` no primeiro `nextval`, e não `last_value + 1`. Comparar só o `last_value` com o máximo deixa passar exatamente esse caso.

3. UNIQUE total impede soft delete

Quatro tabelas têm índice único sem `WHERE deleted_at IS NULL`: `classe_secreta_revelada`, `class_level_progression`, `skill_niveis` e `npc_acesso_player`. Nelas uma linha soft-deletada continuaria ocupando a chave e impediria recriar o registro — e o culpado estaria invisível na listagem. Por isso o DELETE nessas quatro é de verdade.

O caso oposto também morde: `character_creation_requests.username` tinha índice *total*, o que impedia um jogador rejeitado de reenviar. Virou parcial na migration 069.

4. PATCH que substitui o JSONB inteiro

PATCH `/api/personagens/:id` troca `data` por completo. Mandar um objeto parcial apaga atributos, classes, skills e inventário sem erro nenhum. Ler, alterar a chave, devolver o objeto inteiro.

5. Caminho relativo devolvido cru

O banco guarda `personagens/inari.png`; o frontend precisa da URL completa. Toda leitura que expõe imagem precisa passar por `montarUrlPublica`. Esquecer não dá erro — devolve o caminho cru e o navegador busca contra a própria origem, dando 404.

Ficou escondido por muito tempo porque todas as linhas ainda tinham URL completa do Supabase gravada, e a função deixa URL completa passar intacta. Quatro caminhos de leitura estavam errados e só apareceram quando a primeira linha passou a guardar caminho relativo.

6. Promise.all em carregador de tela

Uma dependência que falha derruba a tela inteira, mesmo que as outras tenham respondido. Já corrigido com `Promise.allSettled` em `MasterProgressaoView` e `ClassesView`; vale conferir em qualquer tela que ainda apareça em branco.

7. RLS ligado sem policy

42 tabelas com row level security ativo e quase nenhuma policy. Só funciona porque o papel da aplicação tem `BYPASSRLS`. Trocar o papel da conexão por um sem esse atributo faria a maioria das tabelas parecer vazia — sem erro, só sem dados.

8. Referência sem FOREIGN KEY

Apagar um registro referenciado não dá erro. A limpeza é manual, e está implementada só para a deleção de skill do catálogo — que remove o id de `passados.skill_ids`, `titles.skill_ids` e o nome de `classes.starting_skills`. As skills já concedidas em

`characters.data.skills` **não** são removidas: são histórico pessoal.

`classes.passive_skills` e `signature_skill` não são limpos por ninguém.

Documento gerado a partir do estado real do banco em 7 de setembro de 2026. Fonte do arquivo: `docs/banco-de-dados.html`. Para regerar o PDF, renderize esse HTML com o navegador em modo headless.